一、這一週在學什麼

前幾週我們讀完了感測器：相機拍照片、光達打出點、雷達量速度。可是感測器給的只是原料。這一週開始問：電腦拿到這些原料，怎麼看懂，怎麼知道照片裡哪裡是車、哪裡是人？模型是從例子學的，例子要有標準答案，所以第二節自己動手當一次標註員。第三節是期中複習。

節次	主題	要帶走的一句話
第 1 節	電腦怎麼看見車和人	照片只是數字；模型從例子學，給框、類別、分數；一遠、一暗就漏。
第 2 節	資料標註（上機）	標註是模型的課本：規則要先寫好，所有人照同一份標。
第 3 節	期中複習	11/5（四）上課時間考；概念題與圖表判讀，題目越南文為主、中文對照。

兩個網頁

標註練習台：https://ch-neural.github.io/MITUST-Principles-of-Self-Driving-Cars/week8/label/

期中模擬考：https://ch-neural.github.io/MITUST-Principles-of-Self-Driving-Cars/week8/review/

二、電腦怎麼「看見」車和人

照片對電腦只是一堆數字



左：**nuScenes** 前方相機的一張照片；右：左圖黃框那 **8×8** 個像素，每個像素三個數字。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

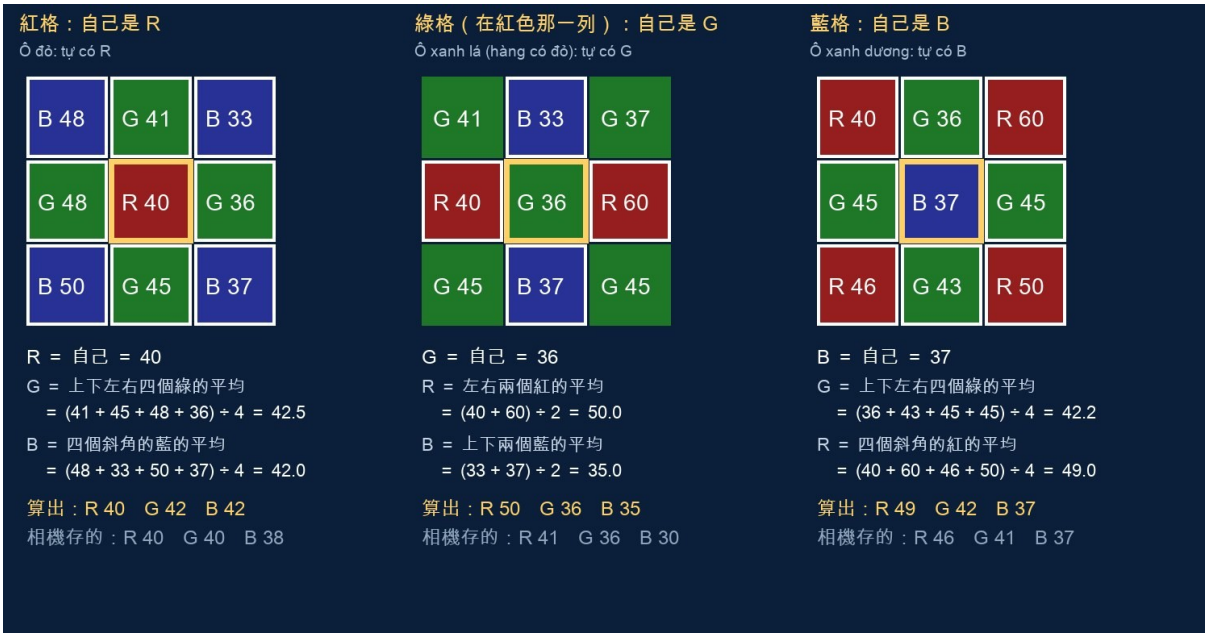
前方相機的照片是 1600 × 900 個像素，每個像素三個數字：紅、綠、藍，各是 0 到 255。整張照片就是 1600 × 900 × 3 = **4,320,000 個數字**，像一張超大的 Excel 表。沒有任何一格寫著「這是人」。電腦要從這四百多萬個數字裡，自己找出哪一塊排起來像一個人。

這些數字怎麼來的：光 → 電 → 數字



同一塊 **8×8** 像素：① 感光格只量得到亮不亮 ② 貼上紅綠藍濾色片，每格只量一種顏色 ③ 用旁邊的格子補齊，每格三個數字。①② 是用這張照片的數字算出的示意。　資料：**nuScenes**（**Motional**），**CC BY-NC-SA 4.0**

相機裡的影像感測器像一大片排整齊的小水桶，每個像素一個。曝光的時候光照進來，光愈多，桶子接到的電愈多，晶片再把電量換算成數字：數字大就是亮，小就是暗。小水桶只分得出光多光少，分不出顏色，所以每一格上面貼一片紅、綠或藍的濾色片（最常見是兩乘兩一組：一紅、兩綠、一藍），每格只量一種顏色。綠色多一倍，是因為人眼對綠光最敏感，照片的亮暗和清晰度主要來自綠色；代價是紅、藍各只有四分之一，顏色細節比較粗，遠處很小的紅燈落到的紅色格子也比較少。缺的兩種顏色，晶片看旁邊的格子估出來補上。存成 JPEG 時，每種顏色用 **8 位元**記，$2^8 = 256$ 級，所以是 **0 到 255**。nuScenes 的相機是 1/1.8 吋 CMOS、1600×900、自動曝光（太暗就多接一點時間），存成 JPEG（Caesar et al., CVPR 2020）。光太強，桶子滿出來，數字頂到 255，那一塊就一片白，叫過曝。

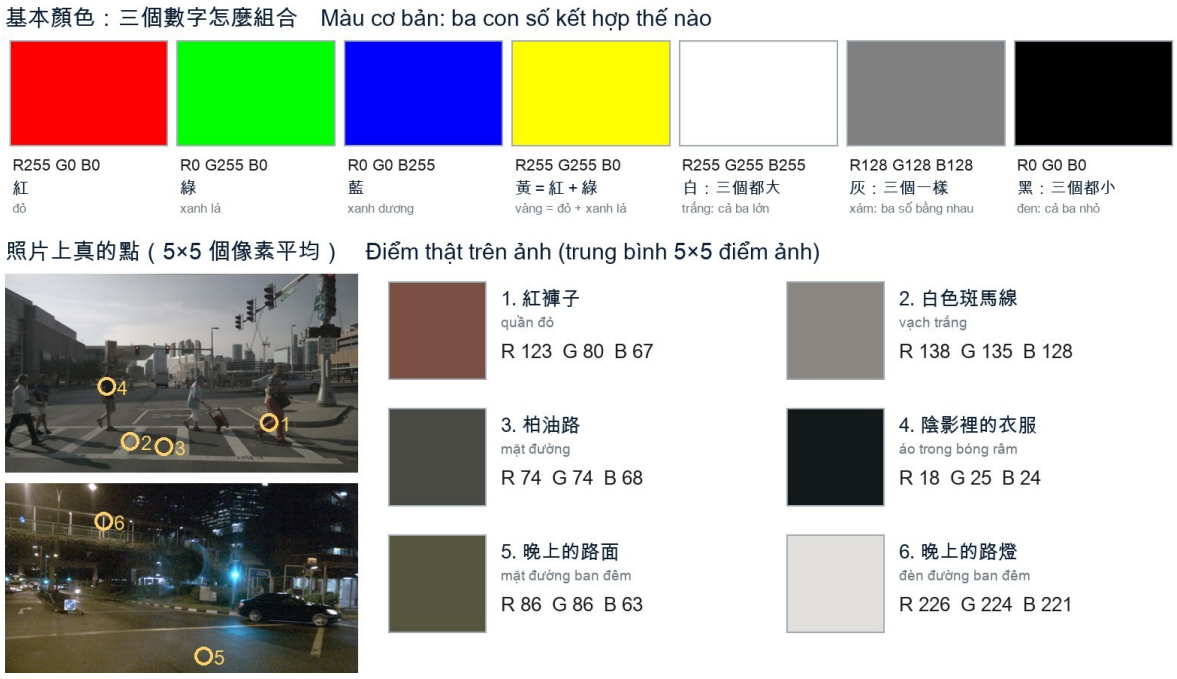


補齊怎麼算（最簡單的鄰居平均）：缺的顏色 = 旁邊同顏色格子的平均。每格那一個顏色的數字取自 JPEG，示意。資料：**nuScenes（Motional）**，**CC BY-NC-SA 4.0**

最簡單的補法是鄰居平均：紅格自己有 R；它的 G 是上下左右四個綠格的平均，例如 (41 + 45 + 48 + 36)÷4 = 42.5；它的 B 是四個斜角藍格的平均。綠格的 R、B 各用左右或上下兩格。真的相機會先看上下、左右哪邊變化小，順著那個方向平均，免得把邊緣兩邊的顏色混在一起（混了會冒出一圈「假色」）；補完還要調白平衡、顏色、亮度，再壓縮成 JPEG，所以和簡單平均算出來的差幾個數。

照片最外面那一圈的格子，旁邊有些鄰居不存在。最簡單的做法是缺的格子不算，用剩下的平均；也可以假裝外面有一圈照鏡子抄過來的格子。真的相機的感光晶片通常做得比照片大一圈，多出來的像素只用來幫忙補顏色，所以照片最外面那一格，在晶片上旁邊其實還有鄰居。受影響的只有最外面一兩圈，對找車、找人幾乎沒有影響。

顏色和數字的關係



上：基本顏色的三個數字；下：照片上真的點（5×5 像素平均），**scene-0553** 白天、**scene-1100** 晚上。　資料：**nuScenes（Motional），CC BY-NC-SA 4.0**

三個數字像紅、綠、藍三支手電筒照在白牆上：只開紅是紅（255, 0, 0），紅加綠是黃，三支都最亮是白（255, 255, 255），三支一樣亮是灰（128, 128, 128），都關掉是黑（0, 0, 0）。讀數字問兩件事：數字大不大（大就亮）、哪一個最大（偏哪個顏色）。照片上的真實點：紅褲子 R 123、G 80、B 67，紅的最大；白色斑馬線只有 138 左右，因為陰天、自動曝光把整張調暗；陰影裡的衣服只有 20 上下，幾乎是黑的；晚上的路面藍的特別小，偏黃，因為路燈是黃光；路燈接近 255。同一種東西換一個光線，數字就差很多，所以電腦不能用「多少到多少是人」的規則去找。

光和顏料的調色不一樣。手電筒自己會發光，每多開一支就多一份光，所以光是愈混愈亮，三支都開就是白色，這叫加法混色。顏料自己不會發光，它把白光裡的一部分顏色吃掉，把剩下的反射出來：黃色顏料吃掉藍光，青色顏料吃掉紅光，兩種混在一起只剩下綠光，所以調出來是綠色。顏料混得愈多，被吃掉的光就愈多，所以愈混愈暗，全部混在一起接近黑色，這叫減法混色。所以印表機用青、洋紅、黃三種墨水再加上黑色；螢幕和相機處理的是光，所以用紅、綠、藍。

為什麼不寫規則：同樣是車，長得都不一樣



nuScenes 裡真的裁下來的車和人：角度、白天晚上、被擋住、遠近都不一樣。「看得到」是官方標註的可見度。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

「車子 = 四個輪子加一塊車身」？從後面看不到四個輪子；晚上只剩兩個車燈；被擋住一半的只看得到車頂。每多一種情況就要多寫一條規則，永遠寫不完。小朋友認車不是背規則，是看了很多台車。現在的電腦也一樣：給它很多有標準答案的照片，讓它自己找出規律，這叫機器學習；模型一層疊一層很深，叫深度學習。

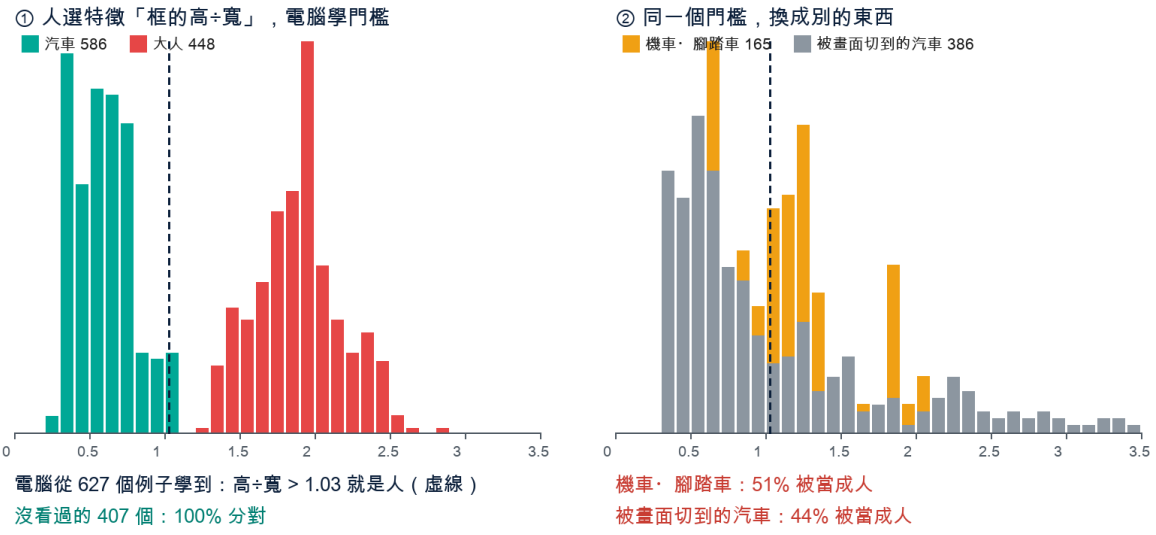
寫規則、機器學習、深度學習：差在誰決定「看什麼」

要認出一個東西有兩個問題：看什麼（照片的哪一種性質，叫「特徵」，例如框是瘦高還是矮寬、顏色、邊的方向）和怎麼判斷（多少以上算人）。三種做法的差別就在這兩個問題由誰回答。

表 1　三種做法比一比

	寫規則	機器學習（傳統）	深度學習
誰決定「看什麼」	人	人	電腦自己從像素學
誰決定「怎麼判斷」	人寫門檻	電腦從例子學	電腦從例子學
找行人的例子	「高÷寬 > 1 就是人」	人選「高÷寬」，電腦學出 1.03；2005 年 HOG（邊緣方向統計）	YOLOX：約 900 萬個數字全部自己學
要多少有答案的資料	不用	幾百到幾千	幾萬以上（COCO 約 12 萬張）
好處	好懂、好檢查	資料少、算得快	最準，人想不到的也學得到

	寫規則	機器學習（傳統）	深度學習
壞處	情況一多寫不完	人選錯特徵就學不好	要大量資料和強的電腦；看不懂它在想什麼



傳統機器學習的例子：人選特徵「框的高÷寬」，電腦從 627 個例子學出門檻 1.03（虛線），沒看過的 407 個全部分對；同一個門檻碰到機車腳踏車，51% 被當成人，被畫面切到的汽車 44%。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

這個例子說明傳統機器學習卡在哪：電腦已經把門檻學到最好了，問題是人選的特徵不夠用，光看高÷寬本來就分不出有人騎的機車和行人。人想得到的特徵有限，每多一種要分的東西就要再想新的。深度學習把這一步也交給電腦。

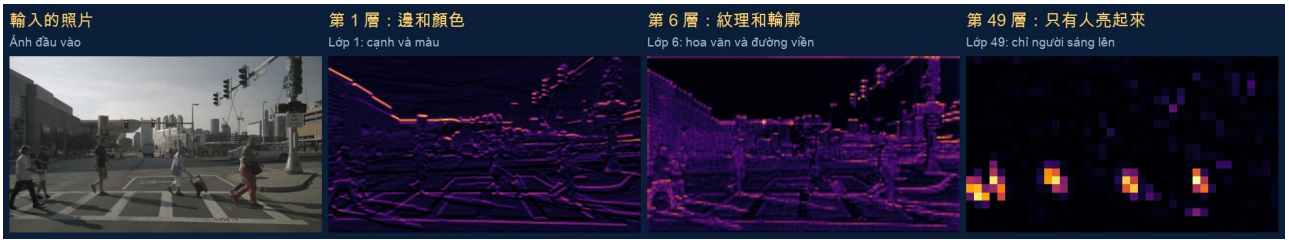


深度學習自己學出來的特徵：YOLOX-s 第一層的 32 個濾鏡（真的數字，每個看 6×6 個像素）：一塊顏色、亮暗交界（邊）、小點。沒有一個是人畫的。偵測模型 YOLOX-s（Megvii，Apache-2.0）。

這 32 個濾鏡不像車，因為它們本來就不是車的零件，而是照片最小的筆畫：就像「車」這個字由橫、豎、點組成，筆畫本身不像車，拼起來才是車。每個濾鏡在照片上一格一格滑過去，每一格問一個小問題：「這一小塊像不像我？」像就亮、不像就暗，得到一張「哪裡有這種筆畫」的地圖。下圖是三個真的例子。



濾鏡怎麼用：三個濾鏡在同一張照片上真的反應（亮 = 像）。紅色濾鏡讓紅褲子亮起來；左暗右亮的濾鏡讓腿、桿子的直邊亮起來；上暗下亮的濾鏡讓斑馬線的橫線亮起來。照片：**nuScenes（Motional），CC BY-NC-SA 4.0**



同一張照片送進 **YOLOX-s**，三個深度真的算出來的結果（亮 = 有反應）：第 **1** 層是線條；第 **6** 層是紋理和輪廓，人、車、牆一樣亮，還分不出人；第 **49** 層只剩行人的位置亮起來。第 **6**、**49** 層各挑一個在人身上最亮的通道。層數是從輸入算起經過幾層，最深 **55** 層。照片：**nuScenes（Motional），CC BY-NC-SA 4.0**

後面的層看的不是照片，而是前一層交出來的那一疊地圖，問的是組合題：「這裡有直邊，右邊又有橫邊嗎？」兩個都有就是一個角，很多直邊和橫邊排在一起就是格子紋。每隔幾層，地圖縮小一半，每一格管的範圍就愈來愈大：第 1 層的一格大約是照片的 5×5 個像素，第 6 層大約 10×10，第 49 層大約 40×40，一格就蓋住半個人。所以後面的層才回答得了「這裡是不是一個人」。

「學」是什麼：猜、對答案、改一點點

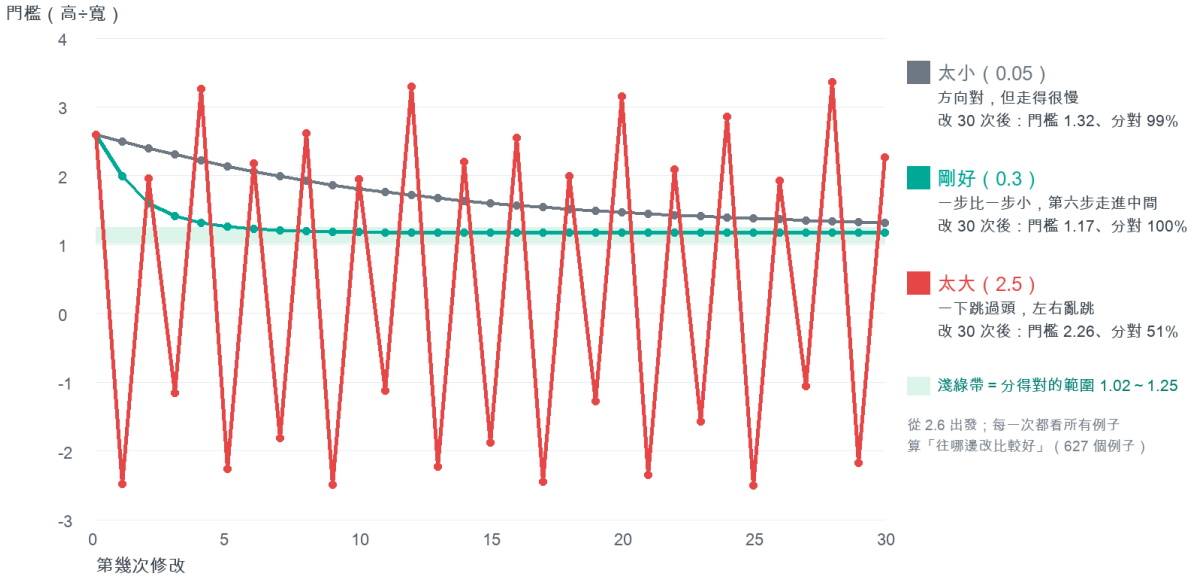
機器學習和深度學習都是這樣學的，都是下面四個步驟，差別只在第三步要改的數字有幾個。

1. 猜：模型先猜一個答案。機器學習的例子：給它一個框，它用高÷寬猜這個框是人還是車；深度學習的例子：丟一整張照片給 YOLOX，它猜「這裡有一個人，框在這裡」。
2. 對答案：跟人標好的標準答案比，看差多少：類別對不對、框偏了多少、有沒有漏掉。
3. 改一點點：把模型裡的數字改一點點，讓下次差少一點。機器學習只改門檻，1 個數字，「看高÷寬」是人選的，電腦不會改；深度學習的 YOLOX 有約 900 萬個數字，連上面那些濾鏡也一起改，所以「看什麼」也是學出來的。
4. 重複：換下一個例子，再猜、再對、再改。幾萬個例子，一輪又一輪。

上面的門檻 1.03，其實不是用這四步學出來的：當時是讓電腦把門檻從 0.5 一路試到 3，每一個都試過，挑分得最準的那一個。門檻只有一個數字，全部試一遍很快；YOLOX 有九百萬個數字，不可能每一種組合都試，只能用這四步一點一點改。下面用同一批例子、改用一點一點改的方法再學一次，結果停在 1.17，一樣全部分對。

就像寫附答案的考古題：寫完對答案，錯的記下來，寫夠多份，沒看過的題目也會寫。第二步的標準答案是人標的，這就是第三章要講的「資料標註」。模型裡面一層一層：前幾層找邊和顏色，中間組成輪子、腿，最後一層說「這是人」。沒有人告訴它「哪些邊是腿」，是它自己在對答案的過程中調出來的。

為什麼只改一點點

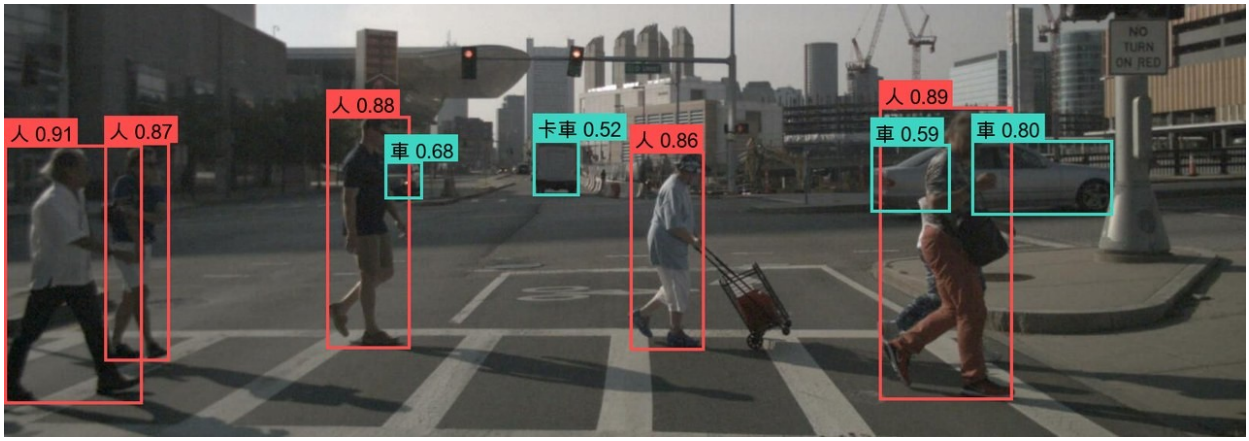


用上面那 627 個例子，讓電腦用「看斜坡、往下走一步」的方式學門檻，三種步伐（學習率）比較。淺綠帶是分得對的範圍。

兩個理由。第一，電腦對完答案只算得出往哪邊改比較好，算不出要改多少，就像大霧裡下山只摸得到腳下往哪邊斜。每一步走多大，是人設定的「學習率」：太小走得慢（0.05 改 30 次門檻才到 1.32）；太大會跳過頭、左右亂跳（2.5 在 −2.5 和 3.3 之間跳，只分對一半）；剛好的話（0.3），一步比一步小（2.6→1.99→1.60→1.41……，愈靠近谷底坡愈平，步伐自己變小），第六次走進分得對的範圍。第二，真的訓練一次只看幾張到幾十張照片，每看幾張就大改，前面從別的照片學到的就被蓋掉了；每次只修一點點，看夠多張，才會對所有照片都好用。

這些東西都有正式的名字。損失函數（loss function）是一條計算規則：給它一個門檻，它算出模型「錯得有多嚴重」的分數；把每個門檻算出的分數連起來，就是那條山谷曲線。它不是數分錯幾個：在淺綠帶裡，分錯的都是 0 個，損失函數還是有高低，1.17 最低，因為它離兩邊最遠、最保險，所以電腦停在 1.17，而不是第 9 張的 1.03。就像射飛鏢，只記有沒有中紅心，不知道要往哪邊修；量離紅心幾公分，才知道偏了多少。腳下往哪邊斜、斜多少，叫梯度（gradient）。梯度指向往上最陡的方向，倒過來的負梯度就是往下最陡的方向。照著負梯度往下坡走一小步、再算一次、再走一小步，一直走到谷底，這個方法叫梯度下降（gradient descent）。合起來說：電腦用梯度下降，照著梯度一步一步走，讓損失函數愈來愈小；每一步走多大，由學習率決定。

物件偵測給三樣東西：框、類別、分數



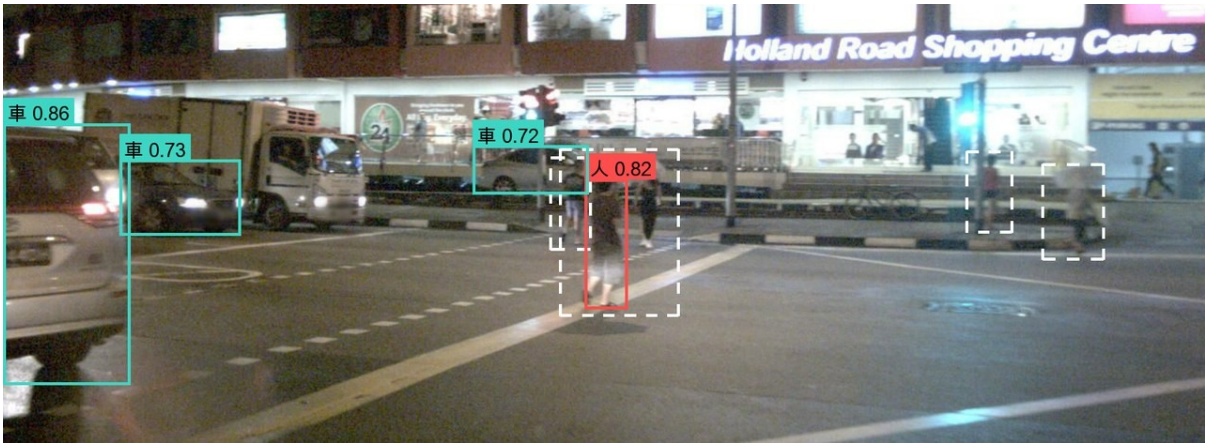
YOLOX-s 在 nuScenes scene-0553 真的跑出來的結果（分數 ≥ 0.5）。偵測模型 YOLOX-s（Megvii，Apache-2.0）；照片：nuScenes（Motional），CC BY-NC-SA 4.0

- 框：東西在照片的哪裡。只知道在照片的左右上下，不知道離車子幾公尺。
- 類別：人、車、卡車……遠處那台「卡車 0.52」其實是公車，類別錯了，分數也比較低。
- 分數：0 到 1，多有把握。0.91 不是「91% 一定對」，它是拿來排序、設門檻的。

表 2　三種常見的視覺工作

	分類	物件偵測	分割
問的問題	照片裡有沒有人？	人在哪裡、有幾個？	每個像素是什麼？
給的答案	整張一個答案	框 + 類別 + 分數	每個像素一個類別
車上的例子	下雨嗎？白天晚上？	車、人、機車、紅綠燈	可以開的路面、車道線

門檻：低了多報，高了漏掉



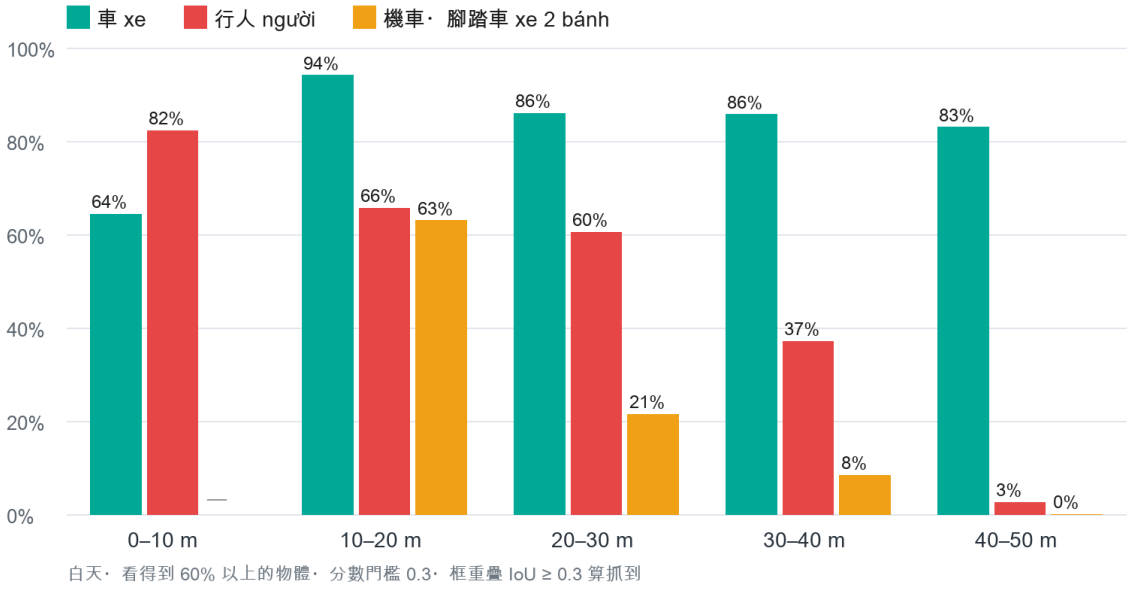
同一張夜間照片，門檻 0.7：框很乾淨，但白色虛線那些行人沒被抓到（官方標註有）。門檻 0.3 時會多抓到幾個，也會多框到商場裡的人。照片：nuScenes（Motional），CC BY-NC-SA 4.0

表 3　nuScenes mini 404 張前方照片：門檻從 0.1 拉到 0.9（看得到 60% 以上、50 m 內）

門檻	框的數量	對不到任何標註的框	車抓到	行人抓到	機車· 腳踏車
0.1	4584	781	87%	63%	38%
0.3	2756	284	82%	53%	27%
0.5	1890	136	74%	43%	15%
0.7	1141	35	59%	30%	8%
0.9	233	0	18%	2%	0%

要多報少，就要接受漏得多。多報，車子為不存在的東西煞車（第七週的幽靈煞車）；漏掉，前面有人卻不煞。漏掉人比較嚴重，所以行人的門檻通常設低一點，多出來的框再靠光達、雷達、追蹤確認。「對不到標註」也不一定都是錯，有些是真的東西只是沒被標（例如店裡的人）。

一遠就漏，晚上也漏

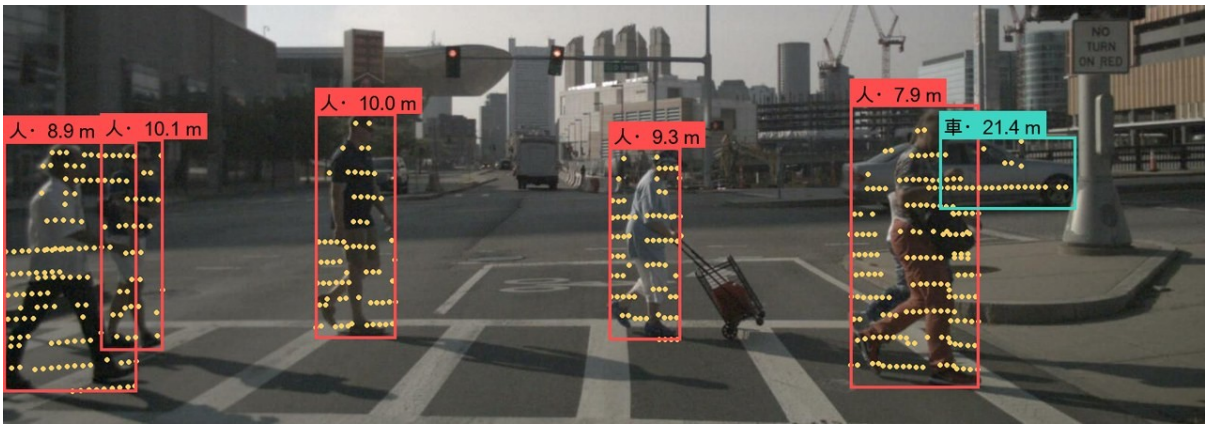


白天 283 張前方照片，看得到 60% 以上、門檻 0.3。行人 10 m 內 82%，40–50 m 只剩 3%；車到 40–50 m 還有 83%。偵測模型 YOLOX-s（Megvii，Apache-2.0）。

1.7 m 高的人：10 m 外約 213 像素高，45 m 外約 47 像素；模型縮到 640 寬後只剩約 19 像素

人和機車小，一遠在照片上就只剩幾十個像素，很難認；車子大得多，掉得慢。晚上也一樣會漏：10–20 公尺的行人，白天抓到 66%，晚上 49%。這就是第七週失效對照表「相機夜間 ▲」的真實數字。

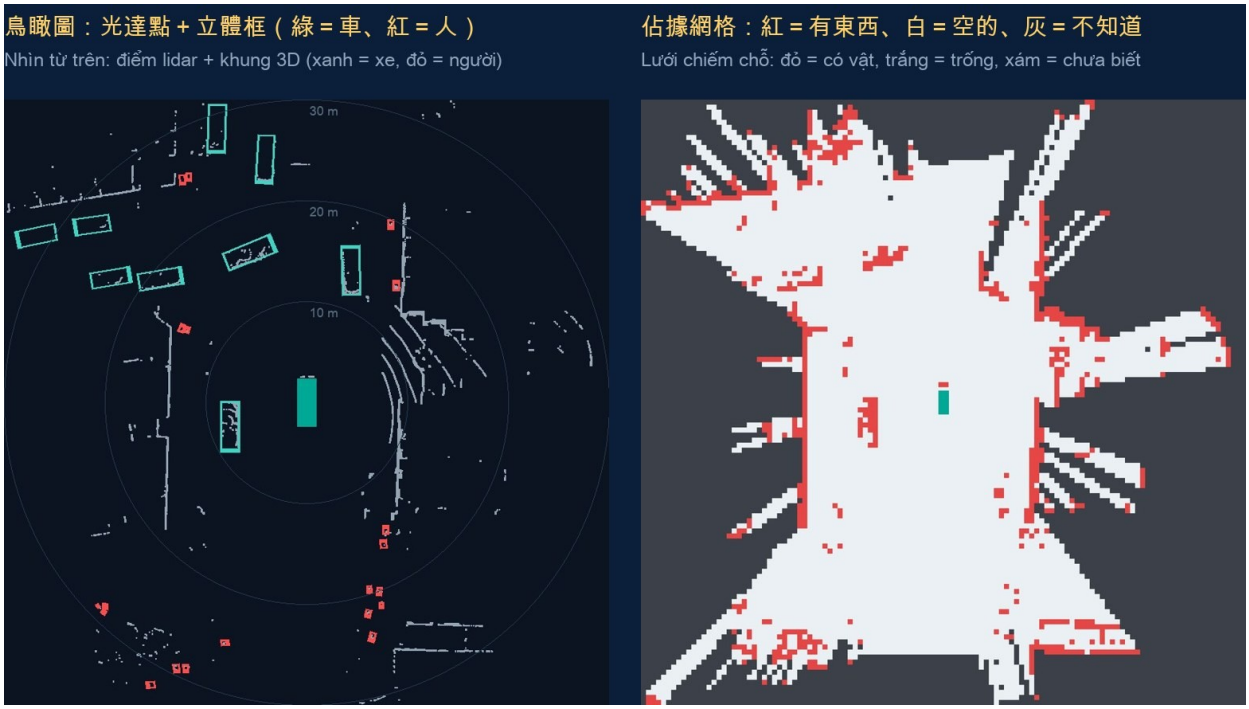
照片的框加上光達的點，才知道多遠



偵測框 + 投到照片上的光達點（黃）。框裡點最密那 1 公尺取中位數：人 8.9 m（官方 9.0 m）、車 21.4 m（官方 22.0 m）。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

相機說「這是人」，光達說「多遠」，這就是第七週的相機 + 光達搭檔。框是長方形，框的角落也會有背後的路面和建築的點，所以要挑框裡點最密的那一群。自駕車做決定要的是立體框：位置三個數字、大小三個數字、加上朝向，共七個數字（第七週）。

從上往下看：鳥瞰圖和佔據網格



左：鳥瞰圖，光達點 + 官方立體框；右：同一格光達算出的佔據網格（0.5 m 一格）。scene-0103（波士頓）。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

- 鳥瞰圖的三個好處：東西不會互相擋；距離不會變形，一公尺就是一公尺；開車本來就是在地面上做決定。第七週的中期融合，就是大家先畫成同一張鳥瞰圖。
- 佔據網格：每一格只問一件事。紅 = 有東西（有高出地面的點）、白 = 空的（雷射穿過沒碰到東西）、灰 = 不知道（被擋住、沒打到）。
- 最大的好處：不用知道那是什麼。掉在路上的貨、沒看過的路障，偵測器認不出來，佔據網格一樣標紅。但要小心：「不知道」不等於「空的」，大卡車後面可能就站著一個人。

延伸觀看（英文影片，可開自動翻譯字幕）

影像感測器：光怎麼變成數字（Branch Education）：youtu.be/B7Dopv6kzJA，7:47–11:18

像素就像接雨水的水桶（CreativeLive，John Greengo）：youtu.be/BsXIQkhsxEM，0:13–1:27

Bayer **濾色片與補齊顏色**（Computerphile）：youtu.be/LWxu4rkZBLw，0:17–5:46

神經網路一層一層（3Blue1Brown **第一集**）：youtu.be/aircAruvnKk，5:32–8:39

梯度下降：球滾下山、步伐（3Blue1Brown **第二集**）：youtu.be/IHZwWFHWa-w，5:12–6:54

YOLO **即時物件偵測**（TED，Joseph Redmon）：youtu.be/Cgxsv1riJhI，0:12–5:48

一個像素一個像素的分割（NVIDIA DRIVE Labs 14）：youtu.be/HS1wV9NMLr8，1 **分** 27 **秒**

夜間行人偵測與主動學習（NVIDIA DRIVE Labs 19）：youtu.be/4aq13pB9s7c，1 **分** 51 **秒**

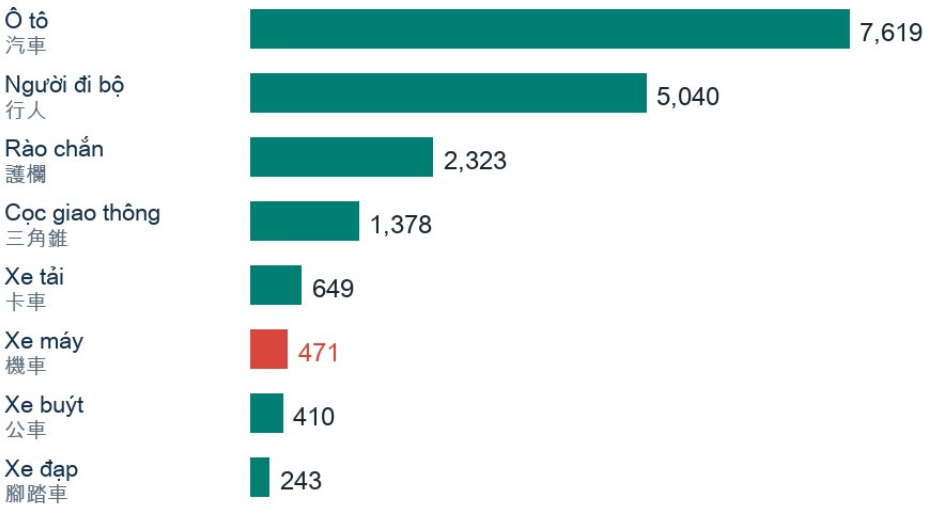
佔據網格五分鐘（Cyrill Stachniss）：youtu.be/8ckhPViqneg，0:04–2:21

三、資料標註：幫電腦準備標準答案

標註資料就是模型的課本。三種狀況會讓模型學壞：漏標（照片裡有人卻沒框，模型學到「這種樣子不是人」）、標錯類別（機車標成腳踏車）、每個人標法不一樣（有人把騎士和機車框在一起，有人分開）。所以正式的標註一定有一份規則，所有人照同一份標，還要抽查。

表 4　nuScenes 標了多少

項目	數字
關鍵畫面	4 萬個（1000 段 × 20 秒，每 0.5 秒標一次）
立體框	140 萬個，人一個一個標
類別	23 類（大人、小孩、汽車、公車、機車、三角錐……）
mini 版：汽車 vs 機車	7,619 個框 vs 471 個框：機車少了十六倍

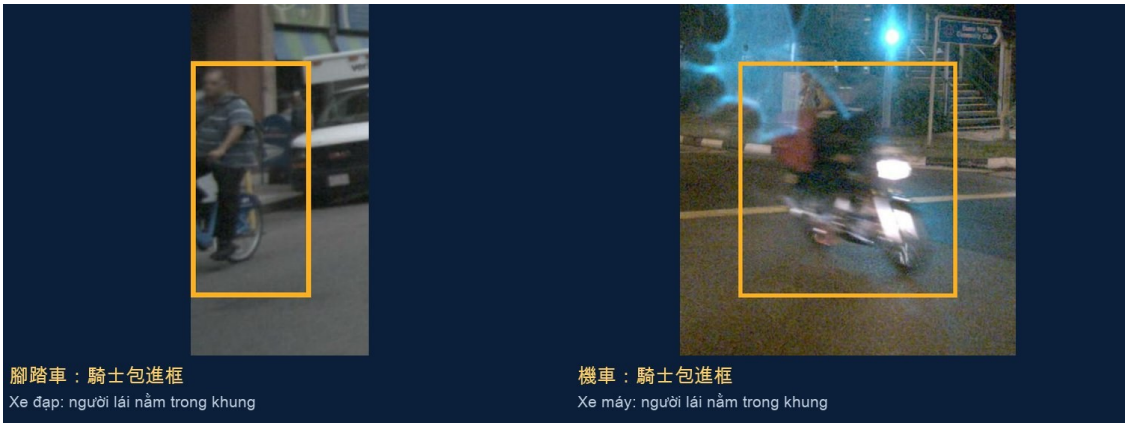


nuScenes mini 各類別的框數（第七週）。波士頓、新加坡的路上機車少，這本課本機車教得很少。　資料：nuScenes（Motional），CC BY-NC-SA 4.0

表 5　nuScenes 官方標註規則（節錄，原文取自 nuscenes-devkit 的 instructions_nuscenes.md）

規則	原文	為什麼
有 1 個光達或雷達點就要標	Label any target object containing at least 1 LIDAR or RADAR point	感測器看得到，模型就該學
框要很貼	Cuboids must be very tight.	框太鬆，位置、距離都不準
手腳要包進去	...should include the extremities.	伸出去的手也會被撞到
有人騎：騎士和車同一個框	If there is a rider, include the rider in the box.	人和車一起動

規則	原文	為什麼
旁邊站的人：另外框	...standing next to the motorcycle, do NOT include...	那個人可能自己走開
可見度分四級	0–40%, 41–60%, 61–80%, 81–100%	知道哪些被擋得多



騎士包在車的框裡：左 **scene-0103** 腳踏車，右 **scene-1100** 機車（官方標註框）。　資料：**nuScenes（Motional）**，**CC BY-NC-SA 4.0**

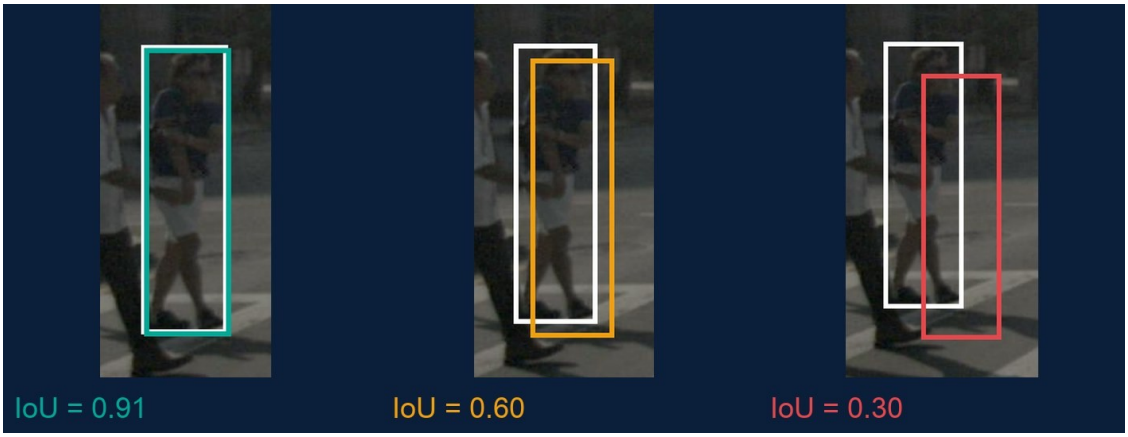
騎士不另外框成行人，他是那台車的一部分。nuScenes 還會在每一台機車、腳踏車上記一個屬性：「有人騎」或「沒人騎」（mini 版的機車：有人騎 176 個框、沒人騎 295 個）。這個屬性只記有沒有人騎，不記騎了幾個人。今天的練習台不用標屬性，把騎士包進框、類別選「機車· 腳踏車」就好。

在河內，一台機車載三個人，照 nuScenes 的規則是一個框，類別是機車，屬性是「有人騎」；載一個人和載三個人，在資料裡看起來一樣。換一套規則把每個人分開框也可以，重點是整份資料用同一套。

延伸觀看

用標註工具 CVAT 畫框（官方教學，英文）：youtu.be/TDae8RkGbKk，0:16–2:47。兩種畫法、調整大小、標「被擋住」。

IoU：兩個框有多像



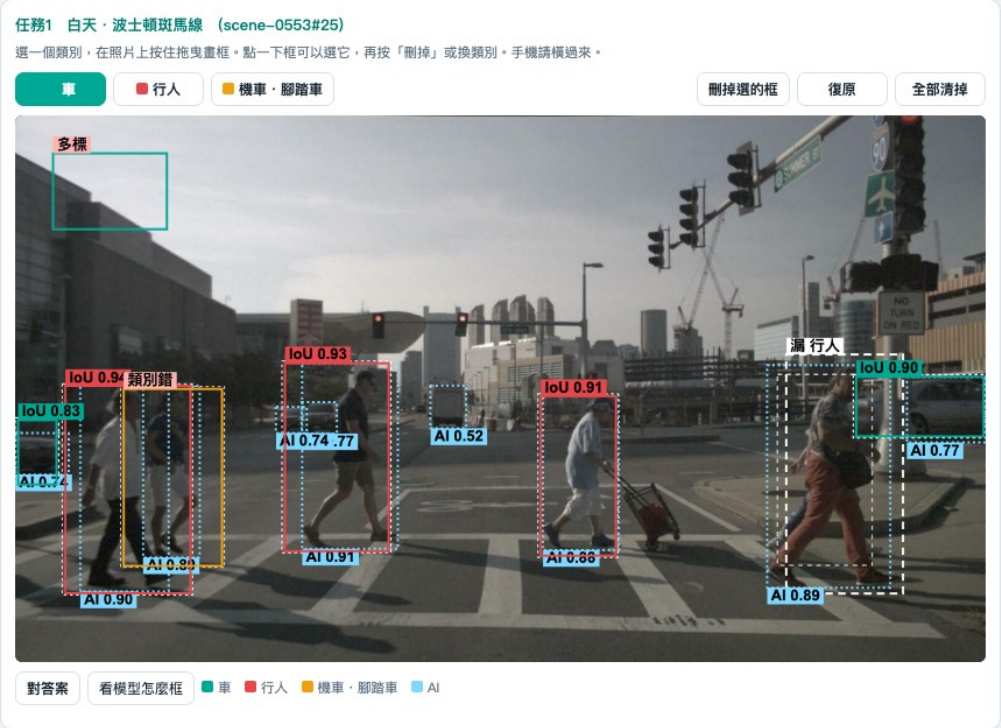
白框 = 參考答案，彩色框 = 示範的學生框（同一個框往右下平移）。　資料：**nuScenes（Motional），CC BY-NC-SA 4.0**

IoU = 重疊的面積 ÷ 兩個框合起來的面積　（1 = 完全一樣，0 = 完全沒碰到）

像兩張紙疊在桌上：重疊的部分除以兩張合起來蓋住的面積。框只偏一點，IoU 就從 0.9 掉到 0.6。標註練習台用 **IoU ≥ 0.3**、類別也對算「框到」，因為參考答案是從立體框算出來的，跟照片上的邊緣本來就會差一些；正式比賽常用 0.5 或 0.7。

為什麼要用 IoU，不直接量框偏了幾個像素？因為同樣偏 10 個像素，對大東西和小東西差很多：200×200 的車框偏 10 個像素，IoU 還有 0.90；20×40 的遠處行人框偏 10 個像素，IoU 只剩 0.33。IoU 用比例算，大小東西可以用同一個標準。IoU 主要用在四個地方：對答案、算成績（第 1 節「抓到幾成」就是 IoU ≥ 0.3 算的；改用 0.5，行人就從 53% 掉到 41%）；去掉重複的框（同一個人有好幾個框，留分數最高的，跟它 IoU 太大的刪掉；YOLOX 設 0.45）；訓練時對答案（看模型的框是在猜哪一個標準答案）；檢查標註品質（兩個人標同一個物體，比一比 IoU）。

上機：標註練習台



對完答案的畫面：框上寫 **IoU** 的是框到了，「多標」是多畫的，白色粗虛線是漏掉的。

1. 打開 https://ch-neural.github.io/MITUST-Principles-of-Self-Driving-Cars/week8/label/，手機請橫過來。
2. 選類別（車、行人、機車· 腳踏車），在照片上按住拖曳畫框。點一下框選它：拖四角或四邊的白色小方塊調大小、拖中間移動（電腦可用方向鍵微調）；也可以刪掉或換類別。框疊在一起時，在同一個地方再點一下，會換選下一個。
3. 一張標完按「對答案」，再按「看模型怎麼框」，跟 AI 比。
4. 四個任務做完，填姓名學號，按「產生作答單」，截圖傳 LINE。

表 6　四個任務：要標幾個、模型框到幾個（YOLOX-s，門檻 0.3）

任務	要標	模型框到
一、白天· 波士頓斑馬線	7	7 / 7
二、陰天· 新加坡停車場	9	8 / 9
三、白天· 波士頓街道	8	6 / 8
四、夜間雨後· 新加坡路口	9	4 / 9

網頁最後一個按鈕「自己的照片」可以上傳自己拍的照片來標，按下載會存成一個文字檔（YOLO 格式：類別、中心 x、中心 y、寬、高，都除以照片的寬高），第十週訓練模型會用到。自己拍路上的照片，不要拍清楚陌生人的臉。

台灣與越南：本地的資料要自己標

nuScenes 這本課本機車教得很少，拿它教出來的模型到河內、台北，碰到的正好是它最不熟的東西。要在當地上路，就要自己收資料、自己標，而且標之前先定好類別和規則。想一想：

- 機車載三個人，算一個框還是四個框？
- 機車後面綁了一大堆貨，貨要不要包進框裡？
- 三輪貨車算機車還是卡車？路邊攤、坐在小板凳上的人算什麼？

四、期中複習（第一週～第八週第 2 節）

- **11/5（四）上課時間**，佔總成績 15%。
- **題型：概念題、圖表判讀。題目越南文為主、中文對照。**
- **模擬考**：https://ch-neural.github.io/MITUST-Principles-of-Self-Driving-Cars/week8/review/（28 題）。答不出來或想複習時，按該題右上角的「解釋說明」，交卷前也能看；說明會直接講出答案，請先自己想過再按。交卷後會列出哪一週錯最多，以及交卷前看過說明的題號。

表 7　八週地圖：每週一句話

週	主題	一句話
1	分級、ODD	等級講的是分工，不是能力。
2	ODD、感測器原理	「至少」是下限；光達：光速×來回時間÷2；雷達只量朝自己的速度。
3	六種感測器	量得到距離≠知道是什麼；距離兩倍，相機誤差四倍。
4	模擬器、場景	時速 50 煞停 20 m，至少看 25 m。模擬是篩子。
5	相機、光達規格	測距要問反射率；差 1 度，50 m 外差 0.87 m。
6	雷達、超音波、衛星、IMU	沒有一種感測器五個情境都贏。
7	失效與融合	每一種都會壞；合起來，比較準的多相信一點。
8	電腦怎麼看見	從例子學；框 + 類別 + 分數；一遠、一暗就漏。

表 8　第一、二週：六個等級

等級	系統做什麼	誰看路、誰接手	例子
L0	警告、瞬間介入	人	AEB 自動緊急煞車
L1	持續做一項：方向或速度	人	ACC 跟車
L2	方向和速度都做	人要一直看路	置中 + 跟車
L3	ODD 內全部自己開	人可以不看路，被要求時要接手	高速公路塞車
L4	ODD 內連接手都自己做	不需要人	區域內無人計程車
L5	沒有 ODD 限制	不需要人	目前不存在

判斷三步驟：瞬間還是持續？管一項還是兩項？誰看路、誰接手？L2 和 L3 差在出事時誰負責；L4 和 L5 差在有沒有 ODD。ODD 六格：道路類型、速度域、交通狀況、天候與光照、地理範圍、人的角色。讀 ODD 條件：「至少」是下限；mm/h 是雨下多大；lx（勒克斯，lux）是照度，照在那裡的光有

多少（辦公室約 300–500 lx、周圍很暗的公共區域照明約 20–50 lx、天快全黑約 3 lx、滿月 0.05–0.3 lx）；開慢一點不會讓不符合變成符合。

表 9　第三週的六種感測器 + 第六週的 IMU：直接量到什麼（有效距離為課堂設定的晴天值）

感測器	直接量到	有效距離	弱點
相機	角度、亮度、顏色	80 m	距離是算的；怕暗、怕雨
光達	距離（飛行時間）	100 m	一遠就稀；不知道是什麼
雷達	距離 + 朝自己的速度	250 m	角度粗；抓不太到人、機車
超音波	距離	4.5 m	只看近處
衛星定位	自己的座標	±2.5 m	不知道前面有什麼；隧道沒訊號
熱像	溫度差	150 m	解析度低
IMU（第六週）	自己的加速度、轉多快	—	會漂移：誤差跟 t^2 變大，不能單獨用

「其他都是算出來的」：直接量到，是硬體本身感受到、量一次就拿到的量；算出來的，要再計算，通常還要加假設。就像機車時速表量的是輪子轉多快，再乘輪胎一圈多長才算出時速，換了小輪胎就不準。相機的距離假設路面是平的；速度要連續看兩次、位置相減、再除以時間（只有雷達一次量到）；IMU 的位置要一路加起來，誤差會累積。看到一個數字，先問它是量的還是算的，是算的再問靠什麼假設。

怎麼量的是第二週教的：光達距離 = 光速×來回時間÷2（除以 2 是因為光走了一去一回；來回 100 奈秒是 15 m）。雷達比較發出去和收回來的頻率差算距離，用都卜勒量速度，但只量得到朝自己靠近或遠離的那一段，橫著開過去的車會量到接近 0。第三週表上超音波 4.5 m 是課堂設定值；第六週 USS6 規格是 5.5 m。

表 10　第四週：煞停距離（反應 0.3 秒、減速度 6 m/s²）和射線至少多長

時速	30	40	50	60	80	100
煞停距離	8 m	13.6 m	20 m	28 m	48 m	73 m
射線至少	10 m	17 m	25 m	35 m	60 m	90 m

$$\text{煞停距離} = v \times 0.3 + v^2 \div 12 \text{（v 用 m/s）；射線至少} = \text{煞停距離} \times 1.25 \text{（本課的餘裕）}$$

第一週說的「時速 50 約 30 m」是用人的反應 1 秒算的，假設不同，兩個都對。第四週還要記：模擬與現實有三種落差（外觀、行為、物理）；里程不是好的測試單位，場景才是；場景分三層：功能（沒

有數字）、邏輯（給範圍）、具體（給一個值）。模擬器打開雨天，射線只剩 0.55 倍長，可是煞停距離不會跟著變短：射線 35 m、時速 50，雨天只剩 19.25 m，比 20 m 短，就不夠了。

表 11　第五、六週：讀規格表最常考的七個數字

數字	意思
OS1：170 m（80%）/ 90 m（10%）	測距看反射率；設計用 90 m
外參差 1° → 50 m 外差 0.87 m	標定要很準，要定期重標
鏡頭 3.6 mm 81°、8 mm 42°	焦距愈長，視野愈窄；由鏡頭決定
雷達 1.6° → 40 m 外 1.1 m；150 m 外 4.2 m	比車道 3.5 m 寬：分不出哪一道
超音波 5.5 m；時速 20 要 9.2 m	開車時不夠用；只在停車場夠用
衛星定位 2.5 m CEP	CEP（Circular Error Probable，圓概率誤差）：一半時間在 2.5 m 內，不是最多 2.5 m
IMU：誤差跟 t^2 成正比	工業級時速 80 撐約 34 秒；隧道 580 秒撐不住

表 12　第七週：失效對照表（✓ 照常、▲ 變差、✗ 失靈、— 不相關）

條件	相機	光達	雷達	超音波	衛星	IMU
大雨、濃霧	✗	▲	✓	▲	✓	✓
夜間	▲	✓	✓	✓	✓	✓
逆光、隧道出口	✗	▲	✓	✓	—	✓
黑車、玻璃、鏡面	▲	▲	✓	✓	—	—
靜止物、頭頂的橋	✓	✓	▲	—	—	—
遠處的行人、機車	✓	▲	✗	—	—	—
被泥巴、冰雪蓋住	✗	✗	▲	✗	✓	✓
高樓窄巷；隧道	✓	✓	✓	✓	✗	▲

- 三種壞法：看不到、看錯、看不到卻說沒事（最危險）。
- 融合的兩個好處：互補、備援。兩個前提：空間對齊（標定 = 內參 + 外參）、時間對齊（同步）。
- 前期、中期、後期：愈早合，留下的資訊愈多，對齊要愈準。後期融合的對應：雷達的目標配相機的框。
- 比較準的多相信一點：IMU 120（±1）、GPS 118（±3）→ (9×120 + 1×118)÷10 = 119.8 m。

- 降級：L2 駕駛馬上接手；L3 請駕駛接手，沒人接就在車道內停；L4 自己靠邊停。

表 13　第八週：電腦怎麼看見，和資料標註

主題	要記住的
照片是數字	每個像素三個數字 R、G、B，各 0–255；紅 + 綠 = 黃，三個都 255 是白
「學」是什麼	猜、對答案、改一點點，重複很多次；步伐太大會跳過頭
機器學習 / 深度學習	機器學習：人決定看什麼；深度學習：連看什麼都自己學。兩種都要有答案的資料
偵測給什麼	框 + 類別 + 分數。門檻低就多報，門檻高就漏掉
一遠、一暗就漏	行人 0–10 m 82%、40–50 m 3%；10–20 m 的行人：白天 66%、晚上 49%
標註、IoU	框要貼、手腳包進去、騎士跟車同一個框。IoU = 重疊÷合起來（合起來要減掉重疊）：1 是完全一樣，0 是沒碰到

表 14　最常講錯的八句話

錯的說法	正確的是
AEB 是 L1	L0：只是瞬間介入
沒有方向盤就是 L5	L5 是沒有 ODD 限制；目前不存在
換掉人就少 94% 車禍	那是事故鏈最後一環，不是唯一原因
測距 200 m 就是 200 m 內都看得到	要問對多少反射率、偵測機率多少
光達不怕天氣	雨霧會讓光達變差；比較不怕的是雷達
雷達不能用來看人	看得到但抓不穩；不能單獨用
兩顆好的合起來一定好	沒標定、沒同步，一樣錯
分數 0.91 就是 91% 一定對	分數是拿來排序、設門檻的

圖表判讀四步驟

1. 看標題和單位：這張圖或表在講什麼、數字的單位是什麼。
2. 看軸和欄：橫軸、縱軸、每一欄各代表什麼。
3. 找前提條件：白天還是晚上、門檻多少、對多少反射率。陷阱常常藏在這裡。
4. 再比較：誰大誰小、怎麼變。